

Нейронные сети Хопфилда на языке Go

🕒 12 минут | 📖 2465 слов 🧑

Продолжая изучать сферу искусственных нейронных сетей (https://en.wikipedia.org/wiki/Artificial_neural_network) (ИНС), я узнаю о действительно крутых типах нейронных сетей, которым в наши дни уделяется не так много внимания. Многие из этих сетей по разным причинам не нашли широкого практического применения в «реальном мире». Тем не менее я нахожу их изучение в свободное время довольно увлекательным занятием, поскольку они часто заставляют меня взглянуть на некоторые проблемы под другим углом. Одна из таких искусственных нейронных сетей называется сеть Хопфилда (https://en.wikipedia.org/wiki/Hopfield_network) (также *сеть Хопфилда*), в честь ее изобретателя Джона Хопфилда (https://en.wikipedia.org/wiki/John_Hopfield).

В этом посте я кратко расскажу о наиболее интересных, на мой взгляд, особенностях сетей Хопфилда. Я часто экспериментирую с тем, что изучаю, поэтому в этом посте я также расскажу о своей реализации сетей Хопфилда на Go (<https://golang.org/>), которую я назвал `gorpfield` (<https://github.com/milosgajdos/gorpfield>). Я снова использовал Go, потому что на данный момент мне удобно писать на этом языке, по крайней мере до тех пор, пока я не освою TensorFlow (<https://www.tensorflow.org/>). Если вы нашли в посте ошибки или он показался вам интересным, не стесняйтесь оставлять комментарии в конце! Давайте начнём!

Человеческая память

Давайте сначала поговорим о человеческой памяти, поскольку некоторые ее функции моделируются сетью Хопфилда. Человеческая память — невероятно интересная и сложная штука. Когда я думаю о ней, мне часто вспоминается рассказ Борхеса (https://en.wikipedia.org/wiki/Jorge_Luis_Borges) под названием «Фунес, памятливым» (https://en.wikipedia.org/wiki/Funes_the_Memorious). Это история о человеке, который помнит все до мельчайших подробностей. Такое состояние также известно как гипертимезия (<https://en.wikipedia.org/wiki/Hyperthymesia>). Чтобы в мельчайших подробностях вспомнить все, что мы пережили, должно требоваться огромное количество энергии — по крайней мере, когда речь идет о воспроизведении воспоминаний. Чтобы получить доступ к памяти, потребуется серьезная «выучка» огромного количества хранящихся в ней данных.

Большинство людей, по-видимому, запоминают в основном «ключевые особенности» того, что с ними происходит. Мы часто забываем или вспоминаем события не такими, какими они были на самом деле. С помощью мозга [нейронов] мы можем собрать воедино впечатления, основанные на ключевых особенностях, хранящихся в нашей памяти, и получить достаточно целостную картину. Эта **сборка** из отдельных фрагментов лежит в основе модели памяти Хопфилда. Давайте рассмотрим ее подробнее.

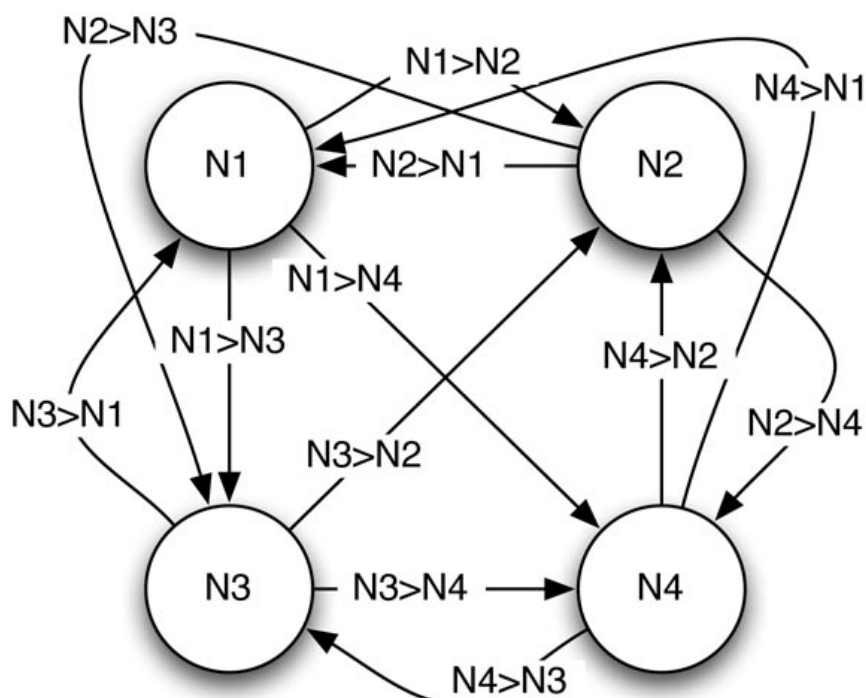
Сеть Хопфилда также можно использовать для решения некоторых задач оптимизации, таких как задача коммивояжера (https://en.wikipedia.org/wiki/Travelling_salesman_problem), но в этом посте я сосредоточусь только на аспекте, связанном с памятью, поскольку он кажется мне более интересным. Кроме того, чтобы использовать сеть Хопфилда для решения задач оптимизации, часто приходится вручную составлять матрицу весов, что довольно трудоёмко и в целом непросто.

Сеть Хопфилда

Сеть Хопфилда — это рекуррентная нейронная сеть (https://en.wikipedia.org/wiki/Recurrent_neural_network), которую можно использовать в качестве контентно-адресуемой памяти (https://en.wikipedia.org/wiki/Content-addressable_memory) (Content Addressable Memory, CAM). Рекуррентные нейронные сети способны сохранять **внутреннее состояние**,

что делает их отличным выбором для моделирования различных функций памяти. Это не значит, что рекуррентные нейронные сети полезны только для моделирования памяти! Их способность сохранять внутреннее состояние (то есть *запоминать закономерности*) делает их полезными в самых разных областях применения, хотя зачастую они гораздо сложнее сетей Хопфилда.

Сеть Хопфилда состоит из **бинарных нейронов**: они могут быть либо включены, либо выключены (0/1 или +1/-1). Нейроны полностью связаны друг с другом в обоих направлениях, то есть нейрон i связан с нейроном j , и наоборот. Каждому соединению присвоен вес. Значение веса соединения одинаково в обоих направлениях между любыми двумя нейронами, например вес w_{ij} равен весу w_{ji} . Весовые коэффициенты сети можно представить в виде матрицы весов. Каждый элемент в определенной строке и столбце матрицы весов представляет собой весовой коэффициент связи между соответствующими нейронами. Из-за симметрии связей матрица весов симметрична, а ее диагональные элементы равны 0, поскольку ни один нейрон не связан сам с собой.

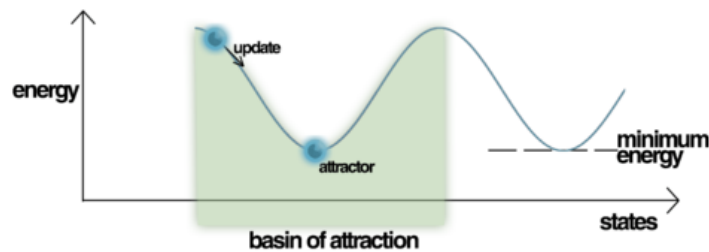


Just like any other ANN, Hopfield network needs to be trained first before it can be used. Unlike some other neural networks, Hopfield network is **trained to store data** (patterns). Since the Hopfield network neurons are binary, the network only allows to store binary data. There are several ways to train Hopfield networks, but I only focussed on understanding the two most well known training algorithms: *Hebbian* and *Storkey* learning.

Hopfield network training stores the binary data [patterns] by modifying the network weights based on particular learning algorithm. Once the network is trained, i.e. it has stored all the data we have supplied to it during training, we can try to recall any of the stored patterns. Before I shed a bit more light on how this process works let's talk a bit about **network energy** which can help us understand how Hopfield networks work.

Energy and pattern basins

Energy is a scalar number (measure) which helps to explain and quantify the network behavior mathematically at any point of time. When we store a pattern in Hopfield network during training, it becomes an **energy attractor**. When we try to recall a pattern from the network by presenting it some data pattern, the stored patterns “attract” the presented data towards them. I will talk about the recollection phase later on, for now just remember that during the pattern recollection the energy of the network changes. If you were to plot the network energy landscape, you would notice that the stored patterns create so called **basins of attractions** to which the newly presented pattern fall in as you try to recall some pattern:



By Mrazvan22 - Own work, CC BY-SA 3.0 (<https://commons.wikimedia.org/w/index.php?curid=23796681>)

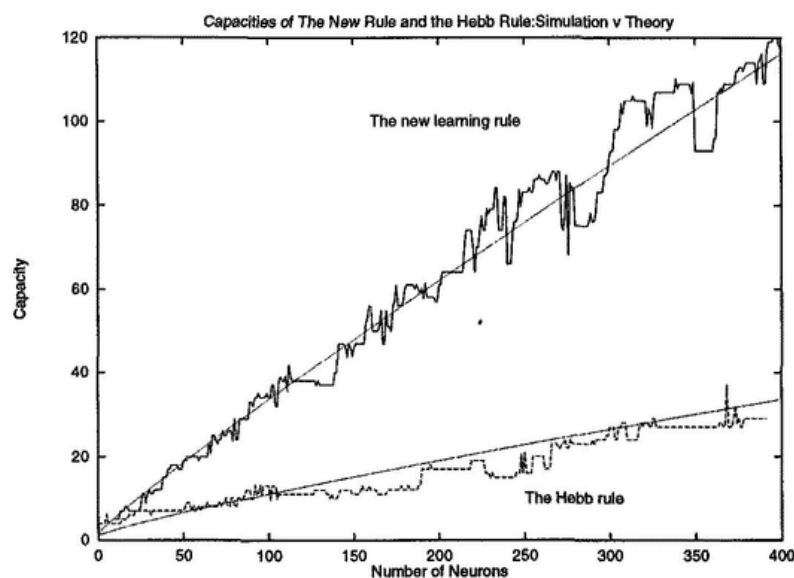
What's really interesting about the energy function (https://en.wikipedia.org/wiki/Hopfield_network#Energy) is that it is a Lyapunov function (https://en.wikipedia.org/wiki/Lyapunov_function). This is a special kind of function which can be used to analyse the stability of non-linear dynamic systems: something I still remember from the control engineering theory I studied at university. For some dynamic systems Lyapunov function provides a sufficient condition of stability. In the context of a Hopfield net, this means that during the pattern recollection the energy of the network is guaranteed to be lowering in value and eventually ends up in one of the stable states represented by the attractor basins. This means that Hopfield network will always recall some pattern stored in it. Unfortunately the attractors represent only local minima so the recalled pattern might not be the correct one. The incorrect patterns are referred to as **spurious patterns** (spurious minima).

Open questions

One of the questions which popped up in my head when learning about Hopfield networks was whether there is some way how to actually make the attractor basin deeper. Intuitively, the deeper the attractor basin, the higher the attraction is. One of the options could be doing something like presenting the same pattern to the network during the training multiple times. This way you could “manipulate” the memory by attracting more patterns to this particular “memory”. Equally, if my theory does work, there should also be a way how to do the opposite i.e. “confuse” the memory by making the deep basins more shallow by training the network with some highly correlated but different data patterns: apparently Hopfield network don't like when the patterns you're trying to store in it are linearly dependent. I'm not sure any of these theories work. I need to do some research and perform some tests on this. I will report back with the results!

Capacity and spurious patterns

Everything in the nature has its limits and so does the storage of the Hopfield networks. The number of patterns a Hopfield network can store is limited by the size of the network and chosen learning algorithm. You can get a rough idea from the graph below



Capacity of Hopfield networks (<http://homepages.inf.ed.ac.uk/amos/capacity.html>)

Storkey learning allows to store more patterns than Hebbian learning, however it takes longer to train the network. Hebbian learning on the other hand is simpler and to me personally easier to understand. Either way, if you exceed the network capacity and try to recall a pattern from it, you might experience the **spurious patterns** I mentioned earlier.

Memory recall

Before we dive into my Go implementation I will shortly describe how the pattern recollection works, because I find it super interesting! During the recall phase the network runs continuously until the states of the neurons in the network settle on some values. Once the energy settles, we are in one of the energy basins. Let's say we have trained our network using one of the learning algorithms. Now, if we want to recall some pattern we present it to the network. Each neuron s_i in the network starts switching its state following the formula below:

$$s_i \leftarrow \begin{cases} +1 & \text{if } \sum_j w_{ij} s_j \geq \theta_i, \\ -1 & \text{otherwise.} \end{cases}$$

where s_j is the state of neuron j , w_{ij} is the connection weight between neurons i and j and θ_i is some threshold, normally set to 0.

We know that the network weights are fixed since they are only modified during training, not during the recall phase. This means that the result of the formula above is driven by the states of the neurons in the network. Notice that a neuron switches its state “on” if majority of the neurons the neuron is connected to are also on. The more neurons are in **on** state, the higher the chance the state of the neurons i will switch on, too and vice versa. This seems sort of like a **peer pressure**. The more of your peers are off, the likelihood you'll switch off, too is very high. How cool is that? :-)

So how can this be useful? Imagine you are a police department and you have a database of criminals. When a new crime happens you interrogate a witness and put together a rough image of the offender. This is not an exact image as its based on the witness description. Now you digitize it and run it against the Hopfield network trained with the images of suspects in that area at the given time. The result can give you an indication of who could be the offender. The idea can be expanded to other use cases, like remembering the passwords from a database of known passwords, an idea I have played around in my head for some time.

Go implementation

Thanks for making it all the way to the Go implementation! Let's dive into some code now! Full source code is available on GitHub (<https://github.com/milosgajdos/gopfield>). The project codebase is pretty simple. It contains a single Go package called `hopfield`. You can create a new Hopfield network like this:

```
n, err := hopfield.NewNetwork(pattern.Len(), "hebbian")
if err != nil {
    fmt.Fprintf(os.Stderr, "\nERROR: %s\n", err)
    os.Exit(1)
}
```

`NewNetwork` function accepts two parameters: the size of the network and type of the training algorithm. The project currently support only the earlier mentioned Hebbian and Storkey algorithms. Now that you have created new network you can start storing some data in it. The data must be encoded as binary vectors since the Hopfield networks can only work with binary data. The `Store` method accepts a slice of `Patterns`.

`Pattern` represents the data as binary vectors. You don't have to craft the `Patterns` manually: `hopfield` package provides a simple helper function called `Encode` which can be used to encode the data into accepted format. The dimension of the data must be the same as the size of the network:

```

pattern := hopfield.Encode([]float64{0.2, -12.4, 0.0, 3.4})
if err := n.Store([]*hopfield.Pattern{pattern}); err != nil {
    fmt.Fprintf(os.Stderr, "\nERROR: %s\n", err)
    os.Exit(1)
}

```

You can store as many patterns in the network as the network capacity allows. Network capacity is defined by both the size of the network and chosen training algorithm. You can query the network capacity or the number of already memorised patterns using the methods shown below:

```

cap := n.Capacity()
mem := n.Memorised()

```

Now that you have stored some patterns in the network you can try to recall some patterns from the network using the `Restore` function. You can specify a mode of the restore (`async` or `sync` - Note: *restore mode has nothing to do with programming execution*) and the number of iterations you want the network to run for:

```

res, err := n.Restore(pattern, "async", 10)
if err != nil {
    fmt.Fprintf(os.Stderr, "\nERROR: %s\n", err)
    os.Exit(1)
}

```

That's all there is to basics. Feel free to explore the full API on godoc (<https://godoc.org/github.com/milosgajdos/gopfield/hopfield>) and don't forget to open a PR or submit an issue on GitHub. Let's have a look at some practical example.

Recovering MNIST images

Apart from the Hopfield network API implementation, the `gopfield` project also provides a simple example which demonstrates how the Hopfield networks can be used in practice. Let's have a quick look at it now! The example stores a couple of MNIST (<http://yann.lecun.com/exdb/mnist/>) images in the Hopfield network and then tries to restore one of them from a "corrupted" image. Let's say we will try to "recover" an image of number **4** which has been corrupted with some noise:



The program first reads couple of MNIST images into memory one by one. It then converts them into binary data patterns using a helper function `Image2Pattern` provided by the project's API. The images are then stored in the network as vectors which have the same dimension as the number of the image pixels: all the MNIST images from particular set have the same dimension which defines the size of the Hopfield network. Go standard library provides a package that allows to manipulate digital images, include gifs! I figured it would be cool to render a gif which would capture the reconstruction process:



You can see how the network pixels are reconstructed one by one until eventually the network settles on the correct pattern which represents the image we were trying to restore. We could also plot the energy of the network over the course of recall and we would see that the energy eventually settles on some value which corresponds to one of the local minimas. There is much more to the implementation than what I briefly describe, so feel free to inspect the actual source code on GitHub. This example hopefully gives you at least some visual context about how the Hopfield network recall process works.

Conclusion

Thank you for reading all the way to the end! I hope you enjoyed reading the post and also that you have learnt something new. We talked about Hopfield networks and how they can be used to memorize and later recall some data.

I have also introduced project `gopfield` which provides an implementation of Hopfield networks in Go programming language. Finally I have illustrated how the project can be used in practice by reconstructing a corrupted MNIST image from the network.

Let me know in the comments if you come up with some cool use of the Hopfield networks or if you know the answers on some of the questions I wondered about in the post.

Tags: `#machine learning` (<https://cybernetist.com//tags/machine-learning/>) `#neural networks` (<https://cybernetist.com//tags/neural-networks/>) `#Go` (<https://cybernetist.com//tags/go/>) `#go` (<https://cybernetist.com//tags/go/>) `#golang` (<https://cybernetist.com//tags/golang/>) `#Golang` (<https://cybernetist.com//tags/golang/>) `#hopfield` (<https://cybernetist.com//tags/hopfield/>) `#Hopfield networks` (<https://cybernetist.com//tags/hopfield-networks/>)

✕

(<https://x.com/share?url=https%3a%2f%2fcybernetist.com%2f2017%2f03%2f12%2fhopfield-networks-in-go%2f&text=Hopfield%20networks%20in%20Go&via=milosgajdos>)

Y

(<https://news.ycombinator.com/submitlink?u=https%3a%2f%2fcybernetist.com%2f2017%2f03%2f12%2fhopfield-networks-in-go%2f&title=Hopfield%20networks%20in%20Go>)

🦋

(<https://bsky.app/intent/compose?text=Check%20out%20Hopfield%20networks%20in%20Go%20https://cybernetist.com/2017/03/12/hopfield-networks-in-go/%20via%20@cybernetist.com>)

😊

(<https://reddit.com/submit?url=https%3a%2f%2fcybernetist.com%2f2017%2f03%2f12%2fhopfield-networks-in-go%2f&title=Hopfield%20networks%20in%20Go>)



([//www.linkedin.com/shareArticle?url=https%3a%2f%2fcybernetist.com%2f2017%2f03%2f12%2fhopfield-networks-in-go%2f&title=Hopfield%20networks%20in%20Go](https://www.linkedin.com/shareArticle?url=https%3a%2f%2fcybernetist.com%2f2017%2f03%2f12%2fhopfield-networks-in-go%2f&title=Hopfield%20networks%20in%20Go))

See also

- [Using Cuelang With Go for LLM Data Extraction \(/2024/05/13/using-cuelang-with-go-for-llm-data-extraction/\)](/2024/05/13/using-cuelang-with-go-for-llm-data-extraction/)
- [Go or Rust? Just Listen to the Bots \(/2024/04/25/go-or-rust-just-listen-to-the-bots/\)](/2024/04/25/go-or-rust-just-listen-to-the-bots/)
- [Circular Buffer Performance Trick \(/2024/04/11/circular-buffer-performance-trick/\)](/2024/04/11/circular-buffer-performance-trick/)
- [A Small Tool for Exploring Text Embeddings \(/2024/03/27/a-small-tool-for-exploring-text-embeddings/\)](/2024/03/27/a-small-tool-for-exploring-text-embeddings/)
- [Fun With AI Embeddings in Go \(/2024/01/07/fun-with-embeddings/\)](/2024/01/07/fun-with-embeddings/)

[← PREVIOUS POST \(HTTPS://CYBERNETIST.COM/2017/01/30/KUBERNETES-SERVICES-AND-INGRESS-UNDER-X-RAY/\)](#)

[NEXT POST → \(HTTPS://CYBERNETIST.COM/2018/03/27/TENSORFLOW-CONDITIONALS-AND-WHILE-LOOPS/\)](#)

SHOW 0 COMMENTS



0

• © 2024 • Cybernetist (<https://cybernetist.com/>)

Hugo v0.136.2 (<https://gohugo.io>) powered • Theme Beautiful Hugo (<https://github.com/halogenica/beautifulhugo>) adapted from Beautiful Jekyll (<https://deanattali.com/beautiful-jekyll/>)